PHYSICAL AI · EDGE INTELLIGENCE

Physical AI
챗봇을 넘어 격전지로

Physical AI: From Chatbots to the Contested Ground

5 Phase 11 Step · 22 산출물 · 실측 데이터 기반 · LED 실습 포함



관측 → 인지 → 판단 → 구동 → 운용 | 2026.10

CHAPTER

1

패러다임 시프트

Paradigm Shift · From Screen to Field

1-1. 화면 안의 AI 지난 3년, AI 는 텍스트를 주고받았다

화면 안의 AI (On-Screen AI)

1

챗봇

Chatbot

사람이 묻고 모델이 답한다. 대화가 끝나면 세상에는 아무것도 남지 않는다.

2

코파일럿

Copilot

사람이 쓰던 것을 이어서 쓴다. 최종 판단과 책임은 사람이 진다.

3

문서 요약

Summarization

긴 것을 짧게 만든다. 요약이 틀려도 원본을 다시 읽으면 된다.

공통점 — **틀려도 되돌릴 수 있다.** 다시 생성하면 그만이다.

1-2. 화면 밖으로 나오면 규칙이 바뀐다

	챗봇형 AI	Physical AI
출력	텍스트	**물리적 상태 변화**
틀렸을 때	재생성	**되돌릴 수 없음**
지연 허용	수 초	**밀리초 (제어)**
입력	사람이 입력	**센서가 계속**
실패 비용	어색한 문장	**기구 파손 · 안전**

“모터는 Ctrl+Z 가 없습니다”

1-3. 왜 격전지인가 먼저 들어간 쪽이 자리를 차지한다

진입 장벽의 세 층 (Three Barriers)

1

구조적 진입 장벽

Structural Barrier

소프트웨어만으로 되지 않는다. 하드웨어·제어·AI 가 동시에 필요하다.

2

현장에만 있는 데이터

Field-Only Data

공개 데이터셋으로 만들 수 없다. 그 공장, 그 조명, 그 카메라의 데이터다.

3

높은 교체 비용

High Switching Cost

설비에 한 번 박히면 수년을 간다. 늦게 들어가면 자리가 없다.

세 장벽은 서로를 강화한다 — **어려워서 늦고, 늦어서 데이터가 없고, 데이터가 없어서 더 어렵다.**

1-4. 왜 지금인가 세 가지가 동시에 가능해졌다

지금을 만든 세 조건 (Enablers)

1

엣지 추론

Edge Inference

8GB 보드에서 비전 모델이 돈다. 오늘 시연한 것이 그것이다.

2

비전·언어 통합

Vision-Language

이미지를 보고 말로 답한다. 규칙으로 못 쓰던 판단이 가능해졌다.

3

온디바이스

On-Device

인터넷 없이, 데이터를 밖으로 내보내지 않고 판단한다.

다음 장부터는 **추정이 아니라 이 보드에서 실제로 잰 값**만 보여드립니다.

PHASE 1 · Observe

STEP 1.2 환경 진단

모델 2.6 GB | **색 판별** 12/12 | **지연** 1~3초

8GB 엣지 보드에서 무엇이 되나

Jetson Orin Nano 8GB · 인터넷 연결 없이 · 실제 측정값

측정 항목	결과
모델	**Qwen2.5-VL 3B (Q4_K_M)**
가중치	**2.59 GB**
잔여 메모리	**2.0 GB**
색 판별 정확도	**12색 전부 한 칸 이내**
추론 지연	**1.3 ~ 3.3 초**
인터넷	**불필요**

왜 3B 인가

7B 는 적재되지만 운용 중 죽는다.
3B 가 더 빠르고 정확하며 유일하게 동작했다.

무엇이 전제인가

GUI 를 끄면 가용이 2.3GB → 6.4GB 가 된다.
데스크톱이 떠 있으면 아무것도 올라가지 않는다.

온디바이스에서 비전 모델이 실제로 돕니다

Phase 1 · Step 1.2

※ 측정 환경: Jetson Orin Nano Super 8GB · JetPack 6 · IMX219 · Qwen2.5-VL-3B Q4_K_M

PHASE 1 · Observe

STEP 1.2 환경 진단

잔여 100 MiB

스왑 3.7GB 전소

그런데 7B 는 죽었다

같은 보드, 같은 모델 계열. 조건만 운용 상태로 바꿨을 때

1 서버 적재

성공

가용 530 MB

→

2 + 카메라 · 제어 루프

견딤

가용 393 MB

→

3 + 추론 요청 1건

사망

스왑 3.7GB 전소

무엇을 놓쳤나

이미지 인코딩에 필요한 연산 버퍼 약 700MB 가 들어갈 자리가 없다. 서버 단독으로는 적재되고, 카메라와 제어 루프까지도 견딘다. 그러나 추론 요청이 들어오는 순간 죽는다.

→ 새 모델을 검토할 때는 반드시 제어 루프를 함께 돌리면서 재야 한다.

적재되는 것과 운용되는 것은 다릅니다

Phase 1 · Step 1.2

※ 측정 환경: 동일 보드에서 카메라 파이프라인·제어 루프를 함께 가동한 상태

PHASE 1 · Observe
STEP 1.1 관측 대상
사례 4건
전부 실제 발생

성공처럼 보였지만 아니었던 것들

네 번 모두 화면상으로는 정상이었다

층위	성공처럼 보인 것	실제
하드웨어	**/dev/video0 존재, 캡처 성공**	전 픽셀이 같은 값 (4100)
모델 서버	**model loaded, /health ok**	첫 요청에 사망
AI 출력	**스키마 통과**	검증기가 5회 중 4회 거부
운용	**30fps 로 잘 돎**	폴백이 실패를 완전히 가림

공통 해법 — **출력을 직접 잰다.** 픽셀 값, 사진 속 색, 실제 요청. 상태 표시가 아니라 결과물을 본다.

무엇을 근거로 “동작한다” 고 말할 것인가

Phase 1 · Step 1.1

1-5. **1부 요약** 네 가지만 기억하면 된다

요약 (Summary)

① **화면 밖**

되돌릴 수 없다.
그래서 검증이 구조여야 한다.

② **격전지**

진입 장벽 · 현장 데이터 · 락인.
늦으면 자리가 없다.

③ **지금 가능**

엣지에서 비전 모델이 돈다.
인터넷 없이.

④ **단, 조건부**

되는 것과 운용되는 것은
다르다.

다음 장 — **그래서 어떤 구조로 만들어야 하는가.**

CHAPTER

2

P-AI 아키텍처 해부

Physical AI Architecture · Core Skeleton

2-1. 전체 구조

여섯 계층과 그 위를 흐르는 다섯 단계

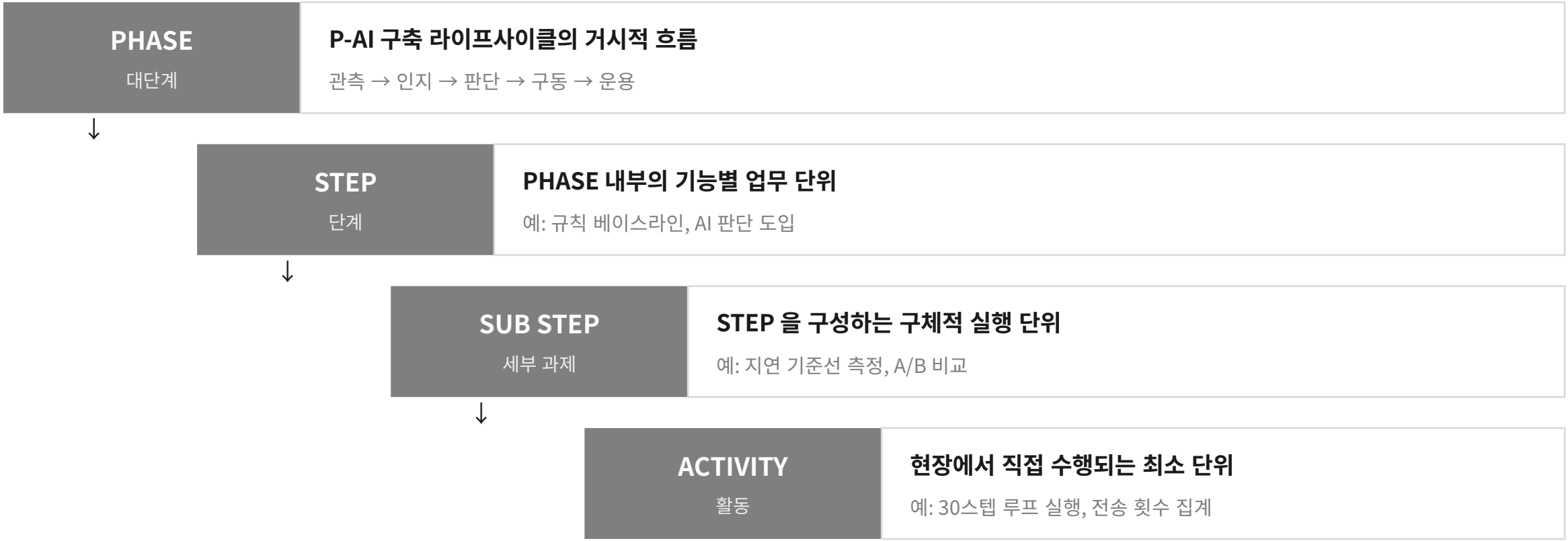
L5	**관측 · 평가**	결정 로그 · 골든셋 · 회귀 탐지
L4	**오케스트레이션**	정책 루프 · 안전 · 기술 기록
L3	**추론**	로컬 VLM · 구조화 출력
L2	**인지**	카메라 · 센서 → 관측
L1	**장치 추상화**	역량 계약 (capability contract)
L0	**하드웨어 · 전송**	MCU · serial / SPI / WiFi

계층 규칙 — **각 계층은 아래 계층 없이 시험할 수 있어야 한다.** 카메라가 없어도 L3~L5 를 개발할 수 있다.

2-2. 수직 구조 거시에서 미시로 쪼개는 계층 축

과제를 시간 순서에 따라 거시에서 미시로 쪼개어, 무엇을 어떤 순서로 수행할지 정의하는 계층 축

수평 구조가 “그 일을 끝까지 가게 하는 힘” 이라면, 수직 구조는 “무엇을 어떤 순서로 하는가” 를 정하는 뼈대입니다.



미시

2-3. 뼈대 ① 느린 판단과 빠른 제어를 분리한다 Physical AI 의 제1원칙

왜 분리해야 하는가 (Why Separate)

1

시간 척도가 다르다

Different Scales

AI 판단은 초 단위, 물리 제어는 밀리초 단위다. 하나로 묶으면 둘 다 망가진다.

2

실패 양상이 다르다

Different Failures

판단은 늦거나 틀릴 수 있다. 제어는 멈추면 안 된다.

3

책임이 다르다

Different Owners

무엇을 할지는 모델이, 어떻게 할지는 MCU 가 정한다.

결론 — **타이밍은 MCU 가 소유하고, 호스트는 의도만 보낸다.**

규칙 베이스라인 — 산출물 예시: 종단 지연 기준선

실측 데이터

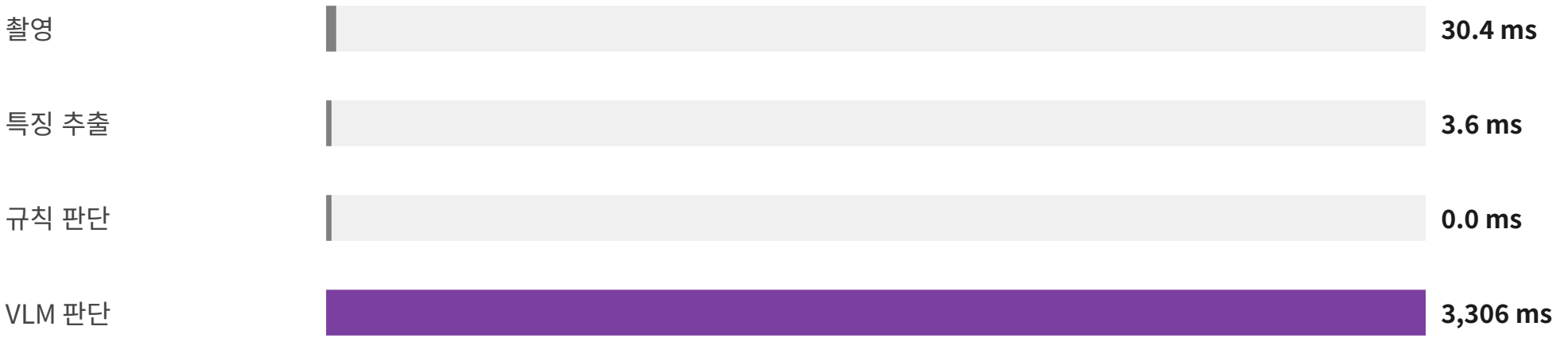
PHASE 3 · Decide

STEP 3.1 규칙 베이스라인

루프 34 ms | **VLM** 3,306 ms | **배율** 97배

판단은 제어보다 97배 느리다

같은 보드, 같은 루프에서 단계별로 나눠 측정



동기로 부르면 0.3 fps · 비동기로 돌리면 30 fps

추론을 작업 스레드로 옮기고 decide() 가 마지막으로 완성된 명령을 즉시 반환하게 하면, 측정된 판단 시간은 0.0 ms 가 된다. 제어 루프는 30 fps 를 유지한다.

모델은 3초 걸리는데 LED 는 끊기지 않습니다

Phase 3 · Step 3.1

※ 측정: argus:0 @640×480, 30~400 스텝 평균. 지연은 단계별로 분리 계측

2-4. 같은 원리가 세 층에서 반복된다

하나의 원리가 세 번 나타난다

분리의 세 얼굴 (Three Faces of Separation)

1

점멸 타이밍

34 ms 아래

호스트가 매번 켜고 끄면 USB 지연과 스케줄러 지터로 떨린다. MCU 가 소유한다.

2

VLM 판단

3초 위

추론을 작업 스레드로 옮긴다. 제어 루프는 마지막 판단을 들고 계속 돈다.

3

출력 흔들림

관측 분해능

센서 노이즈를 그대로 따라가면 LED 가 떤다. 이력으로 붙잡는다.

학습자가 “이거 앞에서 한 얘기 아닌가요” 라고 물으면 — **맞습니다. 그게 핵심입니다.**

2-5. 뼈대 ② 출력은 검증을 통과해야 한다 모델이 지어낸 것이 하드웨어에 닿지 않도록

세 관문 (Three Gates)

1

스키마 강제

Decoding-time

디코딩 단계에서 JSON 구조를 강제한다. 사후 파싱은 반드시 깨진다.

2

계약 검증

Host-side

선언에 없는 장치·명령·범위 밖 값을 거부한다.

3

펌웨어 한계

MCU

같은 범위를 MCU 가 다시 검사한다. 우회 경로가 없다.

중복이 아닌 이유 — **우회 경로가 서로 다르다.** 선언은 안내문이고 펌웨어는 잠긴 문이다.

PHASE 3 · Decide

STEP 3.2 AI 판단

거부율 80%

화면상 30fps 정상

아무것도 고장나 보이지 않았다

모델도 서버도 정상이었다. 계약을 두 곳에 적은 것이 원인이었다

구분	내용	결과
스키마가 말한 것	**colour 는 선택 항목**	모델은 이를 따랐다
검증기가 말한 것	**on/blink 에는 colour 필수**	모델 출력을 거부했다
결과	**5회 중 4회 거부**	그런데 LED 는 계속 켜져 있었다

왜 안 보였나

폴백 정책이 30fps 를 유지했다. 시연 화면은 완벽히 정상이었고,
통계를 열어보기 전까지 아무도 몰랐다.

어떻게 고쳤나

계약을 한 곳에만 적는다. 스키마와 검증기가 같은 선언에서
파생되면 갈라질 대상이 없다. (뼈대 ③)

폴백이 실패를 완전히 가렸습니다 Phase 3 · Step 3.2

※ 발견 경로: policy.stats 의 failed 카운터. 화면만 보고는 알 수 없었음

2-6. “모르겠다” 를 합법으로 만든다

출구를 주지 않으면 모델은 반드시 지어낸다

none 의 설계 (Designing an Exit)

없을 때

모델은 어두운 장면에서도 색을 지어낸다. 확신에 찬 오답이 나온다.

있을 때

“The image is too dark, blurred or ambiguous” — 실제 응답이다.

none ≠ off

“판단할 수 없다” 와 “끄라” 는 다른 말이다. none 은 직전 명령을 유지한다 — 조명이 한 프레임 깜빡였다고 경고등이 꺼지면 안 된다.

모터라면 더 나쁘다 — **판단 불가마다 정지하면 기구가 덜컥거린다.**

2-7. 뼈대 ③ 역량을 데이터로 선언한다 "프로그램 불필요" 가 성립하는 지점

하나의 선언에서 셋이 파생된다 (One Declaration, Three Outputs)

장치 선언

capabilities/*.json

id · kind · transport
commands · args · wire

→

모델 도구 스키마	모델이 고를 수 있는 선택지
출력 검증기	계약 밖 호출을 거부
전송 문자열	wire 템플릿으로 직접 생성

wire 템플릿이 핵심이다 — **이것이 없으면 장치마다 파이썬 어댑터가 필요하고, "선언만 추가" 가 거짓이 된다.**

합격 판정 (Acceptance Test)

새 액추에이터를 추가하는 데 코드를 한 줄도 쓰지 않는다

PHASE 4 · Actuate

STEP 4.1 역량 계약

장치 1 → 2

스키마 3 → 5

코드 0줄

코드 0줄로 장치를 추가했다

선언 파일 하나를 놓은 것 외에 파이썬을 한 글자도 바꾸지 않았다

시점	파이썬 md5	모델이 보는 스키마
servo_pointer.json 추가 전	**51a8999616dcf698…**	옵션 3개 / 장치 1개
선언 파일 1개 추가	**(파이썬 수정 없음)**	—
추가 후	**51a8999616dcf698…**	옵션 5개 / 장치 2개

계약이 막는 것들

laser.fire
× 없는 장치

servo.spin
× 없는 명령

angle = 999
× 범위 초과

torque = 9000
× 없는 인자

환각이 하드웨어에 닿지 않는다. 그리고 범위는 펌웨어가 한 번 더 막는다.

장치 추가에 파이썬을 한 줄도 쓰지 않았습니다

Phase 4 · Step 4.1

※ 검증 방법: 추가 전후 파이썬 파일 전체의 md5 해시를 비교

2-8. AI 를 쓸 문제 / 쓰지 말 문제

문제의 성질이 도구를 정한다

과제	규칙 기반	VLM	승자
색 판별	보통	좋음	**VLM**
위치 판단	9/9 · 7 ms	3/9 · 2.5초	**규칙 (330배 빠름)**
“정리되어 있나”	불가능	가능	**VLM**

차이는 문제의 성질에 있다 — **색은 의미 해석이고, 위치는 산술이다.**

산술을 모델에게 시키지 마십시오

PHASE 3 · Decide

STEP 3.2 AI 판단

규칙 9/9 · 7ms

VLM 3/9 · 2.5초

배율 330배

위치 판단은 규칙이 완승했다

정답이 확실한 합성 이미지 9개. 같은 이미지를 양쪽에 준다

정답 x	정답	규칙	VLM
0.10	**left**	left	left
0.30	**left**	left	left
0.50	**centre**	centre	left
0.70	**right**	right	left

정확도 숫자 하나로 모델을 판단하지 마십시오

답변 목록을 보지 않으면 “좀 부정확하다” 로 읽힌다.
무엇을 못 하는지는 분포에서 드러난다.

실패의 모양

VLM 이 9개 전부 left 로 답했다.
맞은 3개는 우연히 왼쪽이었을 뿐이다.
위치를 조금 틀리는 것이 아니라,
위치를 보고 있지 않다.

이미지를 키우면 나아지나

768×432	**2/5 · 2.3초**
1024×576	**3/5 · 5.6초**
1280×720	**3/5 · 8.1초**

하나 더 맞히는 대가로 지연 3.5배

3/9 만 보면 “부정확”, 답변을 보면 “안 보고 있음”

Phase 3 · Step 3.2

※ 정답은 사각형을 그려 넣은 좌표. 채점에 논란의 여지가 없도록 합성 이미지를 사용

2-9. 2부 요약 뼈대 셋과 원칙 하나

요약 (Summary)

①

느린 / 빠른 분리

Separation

판단은 비동기로, 타이밍은 MCU 가. 97배 차이를 구조로 흡수한다.

②

출력은 검증을 통과

Validation

스키마 · 계약 · 펌웨어. 우회 경로가 달라 중복이 아니다.

③

역량을 데이터로

Declaration

하나의 선언에서 셋이 파생된다. 갈라질 대상이 없다.

그리고 — 산술을 모델에게 시키지 마십시오

CHAPTER

3

P-AI 실습

Hands-on · Colour Control with a Local VLM

인지 구축 — 실습 개요: 카메라가 본 색을 LED 가 재현한다

실습

PHASE 2 · Perceive | STEP 2.1 센서 연동 | 부품 5종 | 소요 60분 | 난이도 중

오늘 만들 것

색지를 카메라 앞에 들면 LED 가 같은 색으로 켜진다

LED	→	카메라	→	로컬 VLM	→	계약	→	LED
빛을 낸다		장면을 담는다		색을 읽는다		명령을 검사한다		다시 켜진다
~1 ms		30 ms		3,306 ms		<1 ms		~5 ms

실습에서 확인할 것

① 색은 어떻게 표현하는가

색상환의 각도 하나. 빨강은 두 번 나온다

② AI 는 얼마나 맞히는가

12색 중 정확 6, 한 칸 이내 12

③ 무엇이 측정을 망치는가

자동보정이 두 방향으로 방해한다

다섯 단계가 모두 이 보드 안에서 끝납니다

Phase 2 · Step 2.1

※ 인터넷 연결 없이 동작합니다. 영상도 판단 결과도 보드 밖으로 나가지 않습니다

PHASE 2 · Perceive

STEP 2.1 센서 연동

함정 4종

손실 3일

이 네 가지를 먼저 확인하십시오

전부 본 커리큘럼 제작 중에 실제로 걸려든 것들이다

함정	증상	확인 방법
Camera Module 3 (IMX708)	**JetPack 미지원. 그런데 /dev/video0 은 생긴다**	레지스터 0x0016 직독 → 0x0708
15핀 CSI 케이블	**꽂히는데 접점이 닿지 않는다 (Orin 은 22핀)**	케이블 끝 금속 접점을 센다
공통 애노드 RGB LED	**RGB 255 255 255 를 보내면 꺼진다**	흰색 명령에 소등되면 공통 애노드
공통 저항 1개	**노랑이 빨강으로, 청록이 초록으로 나온다**	빨강을 줄여 초록이 드러나면 확정

가장 위험한 것은 첫 번째입니다

IMX708 은 IMX477 과 i2c 주소를 공유해서, 오버레이를 올리면 장치 노드가 생기고 캡처까지 성공합니다. 그런데 모든 픽셀이 같은 값입니다.

이 슬라이드가 실습 시간을 절반으로 줄입니다

Phase 2 · Step 2.1

※ 부품을 주문하기 전에 확인하면 가장 좋습니다

PHASE 2 · Perceive

STEP 2.1 센서 연동

판독 레지스터 0x0016

검사 std ≥ 1.0

장치 노드는 드라이버가 만든다

센서가 만드는 것이 아니다. 그래서 칩에게 직접 물어본다

센서 식별 — 칩 ID 판독표

0x02 0x19	**IMX219**	지원 · 주소 0x10
0x04 0x77	**IMX477**	지원 · 주소 0x1a
0x07 0x08	**IMX708**	미지원 · 주소 0x1a

고장난 카메라가 통과시킨 검사들

/dev/video0 존재	✓ 생성됨
드라이버 바인딩	✓ bound
포맷 조회	✓ 3840×2160
프레임 캡처	✓ 성공

건전성 검사 기준

① 모든 픽셀이 같은 값이면 거부
② 표준편차 1.0 미만이면 거부
③ 렌즈 캡을 씌운 어두운 장면은 통과해야 한다

그런데 픽셀을 보니

min = max = mean = 4100, std = 0.0

830만 샘플이 전부 같은 값. 사진이 아니었다.

존재 · 바인딩 · 캡처 성공은 모두 증거가 아닙니다

Phase 2 · Step 2.1

※ 센서는 INCK 클럭이 공급될 때만 i2c 에 응답합니다. 스트리밍 중에 읽어야 합니다

3-1. 질문 — 무지개 색은 어떻게 표현하지? 모델에게 색을 가르치려면 먼저 색을 정의해야 한다

“무지개 색은 어떻게 표현하지?”

가능한 답 — 어느 것이 모델에게 좋은가 (Which Representation)

1

RGB 세 값

Three Channels

사람이 읽기 어렵다. 모델에게 “(255,128,0)” 을 고르라고 하면 범위를 벗어난다.

2

색 이름

Names

어휘가 유한하다. “연두” 와 “초록” 의 경계는 사람도 갈린다.

3

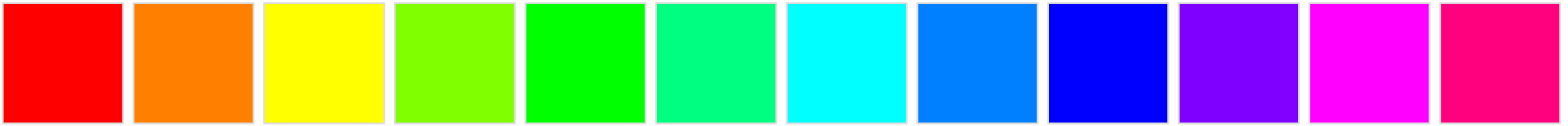
색상환의 각도

Hue

값 하나. 연속적이고 순환한다. 모델은 이름을, 기계는 각도를 쓴다.

3-2. 색은 1차원이다 한 바퀴 도는 숫자 하나

색상환 (Hue Wheel) — OpenCV 는 0–179



0	15	30	45	60	75	90	105	120	135	150	165
빨강	주황	노랑		초록		청록		파랑		자홍	

⚠ 0–179 입니다. 360 이 아닙니다.

OpenCV 는 색상환을 180 단계로 담습니다. 360 을 기대하고 짠 코드는 모든 색을 절반씩 어긋나게 읽습니다.

변환 사슬 — **“green” → hue 60 → RGB(0,255,0) → “RGB 0 255 0 / LEVEL 90 / ON”**

3-3. 빨강은 두 번 나온다 평균을 내면 정확히 청록이 된다

순환하는 축의 함정 (The Wraparound Trap)



hue 2

둘 다 빨강



hue 175

둘 다 빨강

평균 →



hue 88.5

청록

해법

평균이 아니라 최빈값을 쓴다.
히스토그램에서 가장 많이
나온 각도를 고른다.

(2 + 175) ÷ 2 = 88.5 — **빨강 두 개의 평균이 청록이다.** 순환하는 축에서는 산술 평균이 성립하지 않는다.

순환하는 값에 평균을 쓰지 마십시오

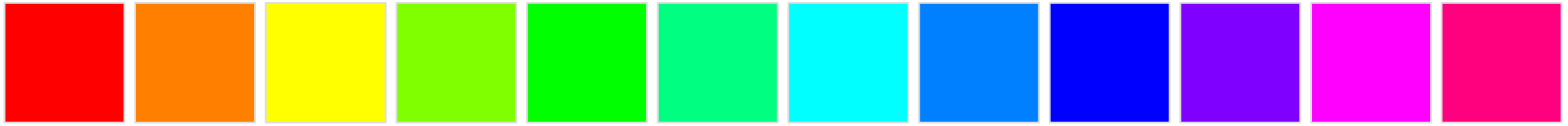
PHASE 3 · Decide

STEP 3.1 규칙 베이스라인

주기 8초 | **단계** 90 | **AI** 사용 안 함

먼저 AI 없이 돌려본다

./host/rainbow.py --period 8 — 색상환을 한 바퀴 도는 것뿐이다



0 15 30 45 60 75 90 105 120 135 150 165

왜 이건 호스트가 타이밍을 쥐나

뼈대 ① 의 의도적 예외다.

① 펌웨어는 색 순환을 모른다. 넣으면 패턴을 바꿀 때마다 재플래싱해야 한다.
② 페이드는 밀리초 정밀도가 필요 없다. 100ms 점멸을 망치던 지터가 여기선 보이지 않는다.

원칙에는 예외가 있다

그리고 예외를 설명할 수 있어야
원칙을 이해한 것이다.

“왜 이건 MCU 가 안 하나요” 라는 질문이
나오면 수업이 잘 된 것이다.

이건 AI 가 아닙니다. 다음 장에서 AI 를 넣습니다

Phase 3 · Step 3.1

※ 카메라로 광학 검증: 색조 37 → 59 → 73 → 95 → 107 → 7 → 10 (한 바퀴)

PHASE 3 · Decide | STEP 3.2 AI 판단

선정 3B | **여유** 2.0 GB | **속도** 2.3배

3B 가 타협이 아니었다

동일 조건에서 제어 루프와 카메라를 함께 돌리며 측정

항목	7B Q4_K_M	3B Q4_K_M
가중치	**5.15 GB**	2.59 GB
색 판별	**측정 불가 (크래시)**	6/6 정확
평균 지연	**2,200 ~ 2,900 ms**	943 ms
최저 가용 메모리	**302 MB**	1,974 MB
동시 부하 결과	**사망**	6/6 성공 · 생존

이 과제에서 7B 의 추가 용량은 **아무 이득이 없었습니다.** 정확도는 같고, 속도는 3B 가 2배 이상 빠르며, 메모리 여유는 6배입니다.

더 빠르고, 정확하고, 유일하게 동작했습니다 — Phase 3 · Step 3.2

※ 측정 조건: 제어 루프 60스텝 + VLM 추론 6회를 동시 실행

PHASE 3 · Decide

STEP 3.2 AI 판단

플래그 4개

전부 필수

플래그 네 개가 결정적이다

그리고 넷 다 직관과 어긋난다

설정	없으면 / 틀리면	비고
--parallel 1	**슬롯 4개 → 첫 요청에서 사망**	KV 캐시가 4배
이미지 768×432	**1280×720 은 batch 초과로 abort**	작으면 인식 정확도 저하
-c 4096	**줄이면 오히려 깨진다**	이미지 토큰이 배치에 들어가야 함
GUI 끄기	**가용 2.3GB → 6.4GB**	multi-user.target

컨텍스트를 줄이는 것이 늘 이득은 아닙니다

이미지 토큰이 배치 안에 들어가야 하므로, -c 를 낮추면 GGML_ASSERT 로 중단됩니다. 메모리를 아끼려다 아예 못 돌게 되는 전형적인 경우입니다.

/health 가 ok 라고 동작하는 게 아닙니다

Phase 3 · Step 3.2

※ 서버는 비전 버퍼 할당에 실패해도 model loaded 를 찍고 포트를 엽니다

PHASE 3 · Decide

STEP 3.2 AI 판단

정확 6/12 | **한 칸 이내** 12/12 | **평균** 3.3초

VLM 은 색을 얼마나 맞히나

색상환을 12단계로 돌며 매번 물었다. 모델은 명령값을 모르고 사진만 본다

색조	견본	명령한 색	모델의 답	판정
0		red	**red**	정확
15		orange	**red**	이웃 한 칸
30		yellow	**yellow**	정확
45		yellow	**green**	이웃 한 칸
60		green	**green**	정확
75		green	**green**	정확
90		cyan	**green**	이웃 한 칸
105		cyan	**blue**	이웃 한 칸
120		blue	**blue**	정확
135		purple	**blue**	이웃 한 칸
150		purple	**purple**	정확
165		red	**red**	정확

정확히 일치

6 / 12

나머지 6개도 전부
색상환에서 바로
이웃한 이름이었다.

지표를 유리하게 고르면

관용 채점하면 12/12 입니다.
그러나 그건 틀린 교훈을
줍니다 — 경계의 이름은
사람도 갈립니다.

“100% 정확” 이 아니라 “한 칸 안에서 정확” 입니다

Phase 3 · Step 3.2

※ 모델: Qwen2.5-VL-3B Q4_K_M · 이미지 768×432 · 화이트밸런스/노출 고정

PHASE 2 · Perceive

STEP 2.2 특징 설계

상쇄 +0.02

증폭 3.2배

같은 기능이 두 방향으로 망친다

있는 것을 감추고, 없는 것을 보이게 한다

전역 밝기로 LED 를 재면

상쇄 **+0.02**

LED 를 최대로 켰는데 화면 평균이 움직이지 않는다. 자동노출이 되돌린다.
→ “카메라가 LED 를 못 본다” 는 틀린 결론에 도달한다.

차분으로 움직임을 재면

증폭 **3.2배**

LED 가 켜지면 자동노출이 화면 전체를 다시 노출한다. 움직인 건 없는데 전 픽셀이 바뀐다.
→ 구름이 지나가도 경보가 울린다.

같은 자동노출이 **모듈 3 에서는 LED 를 지우고, 여기서는 없는 움직임을 만들어냅니다.**

차분 기반 검출에서 노출 고정은 선택이 아니라 전제입니다 Phase 2 · Step 2.2

※ 측정: LED 최대 점등 시 전역 밝기 변화 / 정지 장면에서 LED 전환 시 차분 점수

특징 설계 — 산출물 예시: 노이즈 바닥 측정표

산출물 예시

PHASE 2 · Perceive | STEP 2.2 특징 설계

노이즈 p95 0.300 | 신호 3.245 | 여유 10.8배

임계값은 추측하지 않고 잰다

정지 장면의 노이즈를 먼저 재고, 알려진 변화를 재고, 둘을 가르는 값을 고른다

측정	값
정지 노이즈 p95	**0.300**
LED 전환 신호	**3.245**
여유	**10.8 배**
표본	**정지 59 · 전환 12**

임계값	오경보	검출	
0.601 (p95×2)	0.0%	100%	← 선정
0.901 (p95×3)	0.0%	100%	
1.501 (p95×5)	0.0%	100%	

왜 가장 낮은 값인가

셋 다 오경보 0% · 검출 100% 로 완벽합니다. 그렇다면 더 민감한 쪽이 낫습니다 — 임계값이 낮을수록 작은 변화를 잡아냅니다.
반대로 노이즈 바닥을 재지 않고 추측했다면, 너무 낮게 잡아 오경보가 쏟아지거나 너무 높게 잡아 진짜 변화를 놓쳤을 것입니다.

오경보 0% 를 유지하는 가장 낮은 값을 고릅니다 — Phase 2 · Step 2.2

※ 알려진 변화로 LED 점등/소등을 사용 — 언제 변했는지 정확히 알기 때문

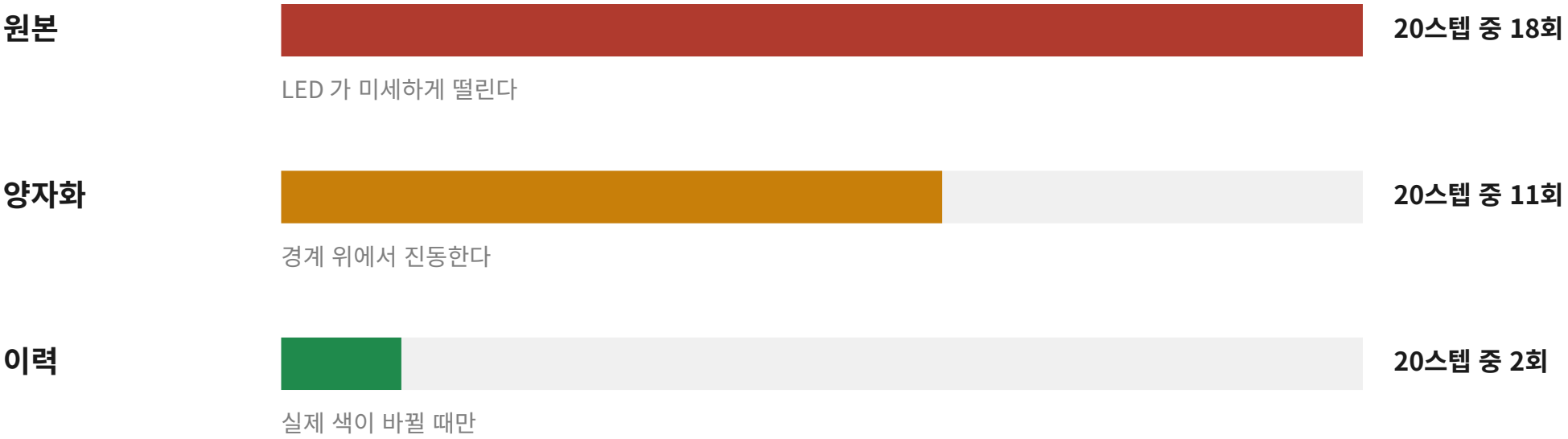
PHASE 3 · Decide

STEP 3.3 출력 안정화

원본 18회 | **이력 적용** 2회

정지한 장면에서 20스텝 중 18회를 보냈다

센서 노이즈로 색조가 흔들리고, 정책이 그것을 충실히 따라간 결과



양자화만으로 부족한 이유 — **값이 스텝 경계 위에서 진동하면 양자화 결과도 함께 진동합니다.** 직전에 내보낸 값과 비교하면 고정된 경계가 없어집니다.

판단의 분해능을 관측의 신뢰도에 맞춥니다 Phase 3 · Step 3.3

※ 장면은 완전히 정지 상태. 변화는 전부 센서 노이즈에서 왔습니다

3-4. 3부 요약 실습에서 배운 네 가지

요약 (Summary)

① **색은 각도 하나**

Hue

0–179. 빨강은 두 번 나오므로 평균이 아니라 최빈값을 쓴다.

② **AI 는 한 칸 안에서**

6/12 · 12/12

정확 6, 이웃 한 칸 6. 한 칸을 넘어 틀린 적은 없었다.

③ **측정이 결론을 만든다**

AE

같은 자동노출이 있는 것을 감추고 없는 것을 만든다.

④ **지표를 고르지 마라**

Metric

관용 채점하면 12/12 다. 그러나 틀린 교훈을 준다.

다음 장 — **그래서 우리 조직은 무엇부터 해야 하는가.**

CHAPTER

4

P-AI 실전 적용

Adoption Roadmap · Where to Start

4-1. 왜 필요한가 조직 관점의 세 가지 동인

도입 동인 (Drivers)

1

사람이 보던 것을 기계가

Automated Inspection

검사 · 감시 · 계수. 사람이 눈으로 하던 반복 판단을 넘긴다.

2

데이터가 밖으로 안 나간다

On-Premise

온디바이스 추론이면 영상도 판단도 보드를 벗어나지 않는다.

3

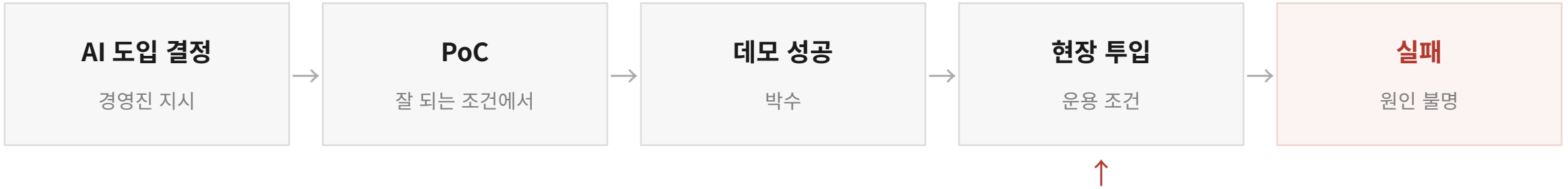
네트워크 없이 돈다

Offline

공장 · 현장 · 오지. 연결이 끊겨도 판단은 계속된다.

그러나 — **“AI 를 넣자” 로 시작하면 실패합니다.** 다음 장에서 그 이유를 봅니다.

4-2. “AI 를 넣자” 로 시작하면 실패한다 흔한 실패 순서



실패 지점 — 데모 환경 ≠ 운용 환경

PoC 는 모델만 띄워놓고 잽니다. 현장에서는 카메라 파이프라인과 제어 루프가 함께 돕니다.
7B 가 적재되고 카메라까지 견디다가 추론 요청에서 죽은 것이 정확히 이 차이입니다 (1-5 참조).

PoC 는 **운용 조건으로 해야 합니다.** 잘 되는 조건에서 재면 잘 된다는 사실만 확인합니다.

측정 조건이 결론을 만듭니다

P-AI 방법론 | 5 Phase · 11 Step | 산출물 22건 | 최소 기간 14주

무엇부터, 어떤 순서로

1단계는 AI 가 아니다 — 지금 사람이 무엇을 보고 있는지부터 적는다

PHASE 1	PHASE 2	PHASE 3	PHASE 4	PHASE 5
관측 설계	**인지 구축**	**판단 설계**	**구동 · 계약**	**운용 · 환류**
무엇을 볼 것인가	센서와 특징	규칙 → AI	장치와 안전	회귀와 확산
관측 목록 · 환경 진단	센서 식별서 · 노이즈 바닥	지연 기준선 · A/B 비교	장치 선언서 · 워치독	골든셋 · 확산 로드맵
2주	3주	6주	3주	지속

가장 흔한 실수 — **Phase 1 과 3 의 규칙 단계를 건너뛰고 바로 AI 로 가는 것.** 비교 기준도, 폴백도, 원인 분리 수단도 없이 시작하게 됩니다.

규칙 단계를 건너뛰면 AI 가 무엇을 개선했는지 알 수 없습니다 — P-AI 방법론

※ 기간은 과제 규모에 따라 달라집니다. 순서는 바뀌지 않습니다

PHASE 1 · Observe

STEP 1.1 관측 대상

기간 2주

질문 하나

“지금 사람이 무엇을 보고 있나”

“AI 로 무엇을 할까” 가 아니다. 질문의 방향이 반대다

사람이 보는 것	빈도	판단 기준	틀리면
부품 색상이 맞는가	**건당**	도색 표준 색상표	재작업 · 고객 반품
작업자가 구역 안에 있는가	**상시**	안전선 기준	안전 사고
조립이 끝났는가	**건당**	부품 6종 체결 여부	후공정 정지
작업대가 정리됐는가	**교대마다**	담당자 판단	다음 조 작업 지연

이 표를 채우면 다음 단계가 자동으로 정해집니다

“판단 기준” 열이 숫자나 규칙으로 적히면 → 규칙으로 됩니다. “담당자 판단” 이라고 적히면 → AI 후보입니다.
“틀리면” 열이 비어 있으면, 그 관측은 애초에 중요하지 않다는 뜻입니다.

사람이 보고 있지 않은 것은 AI 도 볼 필요가 없습니다

Phase 1 · Step 1.1

PHASE 3 · Decide
STEP 3.1 규칙 베이스라인

실측 330배
규칙 판단 7.4 ms

규칙 베이스라인이 주는 세 가지

없으면 AI 를 넣고도 그것이 나아진 것인지 알 수 없다

1. **비교 기준**

   AI 가 얼마나 나아졌는지 재려면 전후를 같은 자로 재야 한다. 기준선이 없으면 “좋아진 것 같다” 밖에 말할 수 없다.

2. **폴백**

   AI 가 죽어도 시스템은 돌아야 한다. 규칙 정책이 그 자리를 메운다.

3. **원인 분리**

   AI 가 이상하게 굴 때 모델 문제인지 배선 문제인지 가른다. 같은 배선에서 규칙이 돌면 배선은 정상이다.

그리고 위치 판단처럼, 규칙이 AI 보다 330배 빠르고 정확한 경우가 실제로 있습니다.

규칙으로 되는 것은 규칙으로 하십시오

Phase 3 · Step 3.1

※ 위치 판단 실측: 규칙 9/9 · 7.4 ms vs VLM 3/9 · 2,472 ms

4-3. 무엇부터 준비해야 할까 영역별 준비물과 흔한 착각

영역	준비할 것	흔한 착각
사람	**측정할 줄 아는 사람**	“AI 전문가부터 뽑자”
데이터	**현장 이미지 + 기대 출력**	“데이터만 많으면 된다”
하드웨어	**실제 운용 조건의 보드**	“일단 좋은 GPU 부터”
프로세스	**골든셋 · 회귀 탐지**	“잘 되면 끝”

가장 중요한 것은 첫 줄입니다 — **측정할 줄 아는 사람이 없으면 나머지가 다 있어도 데모에서 멈춥니다.**

PHASE 4 · Actuate

STEP 4.2 안전 설계

기본값 꺼짐

모터 필수

안전장치는 부품보다 먼저 만든다

붙이고 나서 만들면, 그 사이의 실수 한 번이 기구를 부순다

장치	호스트가 죽으면	안전 상태	위치독
LED	**계속 켜져 있다**	소등	선택
서보	**마지막 각도를 밀어붙인다**	— 환경이 정한다	필수
스테퍼	**계속 돈다 · 기구 파손**	코일 전원 차단	필수
히터	**계속 가열된다**	즉시 차단	필수

⚠ 서보의 안전 상태는 자명하지 않습니다

중립으로 복귀 → 이동 경로에 사람 손이 있으면 그것을 칩니다. 현재 위치 유지 → 부하가 걸려 있으면 과열됩니다.
토크 해제 → 중력에 의해 팔이 떨어집니다. 아래에 무엇이 있느냐가 답을 정합니다.

서보의 “중립 복귀” 가 더 위험할 수 있습니다

Phase 4 · Step 4.2

※ 안전 상태는 장치가 아니라 설치 환경이 정합니다. 선언에 적고 펌웨어가 구현합니다

PHASE 5 · Operate

STEP 5.1 회귀 탐지

사례 5건

통과 2 / 5

2/5 를 그대로 두었다

느슨하게 고쳐 5/5 로 만들면 정확히 틀린 반사신경을 가르치게 된다

사례	기대	실제	판정
red LED	**on · red**	none · red	FAIL
green LED	**on · green**	on · green	PASS
blue LED	**on · blue**	none · blue	FAIL
yellow LED	**on · yellow**	off · yellow	FAIL
LED 소등	**off**	off · white	PASS

느슨하게 되돌리고 싶은 충동이 들 때

묻습니다 — “5/5 로 만들면 모델이 좋아지나요?”

무엇을 찾았나

이 모델은 색은 안정적으로
맞히지만 행동 선택이
일관되지 않다.
같은 장면에서 on / none / off 를
오간다.

부분 일치는 양날이다

적어둔 필드만 검사한다.
적지 않은 필드의 회귀는 조용히 통과한다.

무엇을 고정할지가 곧
무엇을 지키겠다는 선언이다.

골든셋은 통과하라고 있는 게 아니라 알아차리라고 있습니다

Phase 5 · Step 5.1

※ colour 만 고정했을 때는 5/5 통과였습니다. action 을 함께 고정하자 2/5 가 됐습니다

4-4. 다음 주 월요일에 할 한 가지 하나만 고릅니다

선택지 (Pick One)

☐ **① 관측 목록 쓰기**
우리 현장에서 사람이 눈으로 보는 것 5가지를 적는다

☐ **② 규칙 가능성 표시**
그중 규칙으로 되는 것에 표시한다

☐ **③ 운용 조건으로 재기**
보드 하나를 주문해 운용 조건에서 측정한다

내가 고른 것

4-5. 클로징 세 문장으로 끝냅니다

① **화면 밖에서는 되돌릴 수 없다**

그래서 검증이 구조여야 한다

② **측정하지 않은 것은 모른다**

데모와 운용은 다르다

③ **AI 가 항상 낫지 않다**

산술을 모델에게 시키지 마라

저장소 · 실습 자료 · 실측 기록 전체 공개

CHAPTER

5

실습 워크시트

Practice Workshop · Design Your Own

실습 워크숍 | 30분

인원 4~5인 | **산출** 워크시트 2종

한빛정밀(주) 조립 1라인

소형 전자부품 조립 · 2교대 · 작업자 12명

항목	현황
생산	**일 2,400개 · 2교대**
인원	**조립 8 · 검사 2 · 반장 2**
검사	**전수 육안 · 1인당 1,200개**
불량	**약 2.1% · 대부분 색상·체결**
기록	**종이 체크시트 · 수기 집계**

현장의 목소리

검사 담당

"눈이 금방 피로해집니다. 교대 끝 무렵이 특히 걱정됩니다."

반장

"불량이 왜 늘었는지 물으면 답할 자료가 없습니다."

공장장

"사람을 늘릴 수는 없습니다. 6개월 안에 숫자로 보여줘야 합니다."

◆ **작업 힌트**

"눈이 피로하다" 는 관측 빈도 문제입니다. "답할 자료가 없다" 는 기록 문제입니다. 둘은 다른 과제입니다.

30분 안에 "무엇을 AI 로 하지 않을지" 까지 정합니다 Chapter 5

※ 교육용 가상 사례이며 수치는 실습을 위한 가정입니다

실습 워크숍 | 30분 · 3단계 | 후보 5개 | 선택 1개

다섯 후보 중 하나를 고릅니다

과제 발굴 시간을 줄이기 위해 후보를 미리 제시합니다

#	과제	해결할 문제	규칙으로 되나	먼저 따져볼 점
1	**부품 색상 검사**	색상 불량 전수 육안	◯ 색상환으로 가능	조명 편차
2	**작업자 구역 이탈**	안전선 침범 감지	◯ 위치는 산술	오탐 허용치
3	**조립 완료 판정**	부품 6종 체결 확인	△ 일부만	가림 · 각도
4	**작업대 정리 상태**	교대 인수인계 분쟁	× AI 필요	기준의 주관성
5	**이상 움직임 감지**	비정상 동작 조기 발견	◯ 차분으로 가능	노이즈 바닥

실습 진행 (30분)

① 관측 + 과제 선택 10분 | ② 규칙 / AI 판정 15분 | ③ 한 줄 발표 5분

규칙으로 되는 것에 AI 를 쓰면 느리고 부정확해집니다

※ 교육용 가상 사례이며 수치는 실습을 위한 가정입니다

① 관측 목록 · 과제 선택

추진과제	

사람이 눈으로 보는 것을 적고, 그중 하나를 고릅니다 (10분)

사람이 보는 것	빈도	판단 기준	틀리면 생기는 일	선택

고른 과제와 이유

◆ **“판단 기준” 이 숫자나 규칙으로 적히면 규칙 과제, “담당자 판단” 이면 AI 후보입니다.**

“틀리면 생기는 일” 이 비어 있으면 그 관측은 애초에 중요하지 않다는 뜻입니다. 지우십시오.

② 규칙 / AI 판정

추진과제

이 실습의 핵심입니다. 판단 항목마다 규칙으로 되는지 먼저 따집니다 (15분)

판단 항목	규칙으로 되나	근거 (무엇을 재면 되나)	결론

◆ 색은 의미 해석이고 위치는 산술입니다. 산술을 모델에게 시키지 마십시오.

“근거” 열에 무엇을 재면 되는지 쓸 수 있으면 규칙 과제입니다. 쓸 수 없으면 AI 후보입니다.

② 규칙 / AI 판정 · 작성 예시

본 커리큘럼의 실측값으로 채운 예시입니다

추진과제

작성 예시

판단 항목	규칙으로 되나	근거 (무엇을 재면 되나)	결론
부품 색상이 맞는가	△ 조건부	색상환 각도. 단 조명이 고정돼야 한다	규칙 + 노출 고정
작업자가 구역 안인가	○ 된다	가로 위치 좌표. 실측 9/9 · 7.4 ms	규칙 (AI 는 3/9)
조립이 끝났는가	△ 일부	부품 6종의 유무. 가림이 변수	규칙 먼저, 실패분만 AI
작업대가 정리됐는가	× 안 된다	“정리” 를 픽셀로 정의할 수 없다	AI 필요
이상 움직임인가	○ 된다	프레임 차분. 노이즈 바닥 0.300	규칙 (임계 0.601)

다섯 중 넷이 규칙으로 되거나 조건부로 됩니다. AI 가 꼭 필요한 것은 하나뿐입니다.

그 하나(“정리되어 있나”)가 AI 를 쓰는 이유이고, 나머지 넷은 AI 를 쓰지 않는 이유입니다.

③ 한 줄 발표

팀당 1분. 고른 과제와 " AI 로 하지 않기로 한 것 " 을 말합니다 (5분)

추진과제

우리가 고른 과제는

"___ 를 ___ 로 판정한다"

규칙으로 하기로 한 것 — 그리고 무엇을 재면 되는가

AI 로 하지 않기로 한 것 — 그리고 그 이유

가장 중요한 것은 세 번째 칸입니다 — 모든 것을 AI 로 하겠다는 계획은 아무것도 측정하지 않았다는 뜻입니다.

가져가는 양식

현장에 돌아가 과제를 실제로 진행할 때 채웁니다

추진과제

실습 외

안전 설계 — 액추에이터를 붙이기 전에

장치	호스트가 죽으면	안전 상태	워치독

◆ 안전 상태는 장치가 아니라 설치 환경이 정합니다

서보의 “중립 복귀” 는 이동 경로에 사람 손이 있으면 위험하고, “위치 유지” 는 부하가 걸리면 과열되며, “토크 해제” 는 팔이 떨어집니다. 아래에 무엇이 있느냐가 답을 정합니다.

골든셋 — 나빠진 것을 자동으로 알려면

사례명	입력	기대 출력	지키는 것

◆ 적지 않은 필드는 회귀가 조용히 통과합니다

“지키는 것” 을 반드시 적으십시오. 몇 주 뒤 실패했을 때 무엇을 지키려 했는지 알아야 고칠 수 있습니다. 사례 수를 늘리는 것이 품질을 뜻하지 않습니다.

30분 실습에서는 이 둘을 쓰지 않습니다. 개념만 잡고, 현장에서 채웁니다.

안전 설계는 Phase 4, 골든셋은 Phase 5 의 산출물입니다 — 둘 다 4장에서 예시를 봤습니다.

부록 A. 실측 데이터 색인 슬라이드에 쓰인 모든 수치의 출처

수치	출처 문서
7B 사망 · 3B 2.6GB	tech/notes/vlm-on-orin-nano.md
34ms / 3,306ms 지연	tech/notes/vs2-latency-baseline.md
규칙 9/9 vs VLM 3/9	tech/notes/direction-rule-vs-vlm.md
노이즈 0.300 · 증폭 3.2배	tech/notes/motion-threshold.md
전역 밝기 +0.02	tech/notes/led-camera-detection.md
공통 애노드 · 공통 저항	tech/notes/rgb-led-calibration.md
12색 6/12 · 한 칸 이내	visuals/vlm-colour-loop.html
코드 0줄 장치 추가	tech/adr/0010-declare-actuators-as-data.md
전송 18 → 2회	tech/adr/0007-quantise-decisions-to-observation-quality.md

※ 모든 수치는 Jetson Orin Nano Super 8GB 한 대에서 직접 측정한 값입니다.

부록 B. 시연 대본과 축약판 실패 시 대안과 시간별 구성

시연 대본 (Demo Script)

오프닝	vs2_demo.py --policy vlm	설명 페이지 재생
2-3	호스트 ↔ MCU 점멸 비교	녹화본
3-3	rainbow.py --period 8	녹화본
3-5	색지를 바꿔 들기	골든셋 이미지

시연 15분 전 점검

ls /dev/video0 /dev/ttyACM0
make test
./host/evaluate.py ← health 만 보면 모릅니다

축약판 구성 (Short Versions)

60분 18장

오프닝 · 1-2 · 1-5 · 2-3 · 2-8 · 3-1~3 · 4-2 · 4-4

90분 28장

위 + 계약(2-5·2-6) · 측정(3-4) · 준비(4-3)

105분 60장

전체

60분에서도 빼지 않는 것

7B 사망 (1-5) · 위치 실패 (2-8)